

`## Hospital Management System (HMS)`

`## Technical Specification and Tech Stack`

Document date: February 17, 2026

Scope baseline: repository state in 'c:\xampp\htdocs\Hospital_Management_System'

`## 1. System Overview`

HMS is a role-based, multi-module web application for hospital operations. It supports Admin, Doctor, Nurse, Nurse Aid, Reception, Pharmacy (including Senior Pharmacist), and IT Support workflows.

Primary design goals:

- Centralized patient flow from registration to consultation and pharmacy.
- Role-based access control and protected API endpoints.
- Near real-time dashboard refresh across roles.
- Operational observability through health checks and audit logs.

`## 2. Architecture`

HMS follows a modular monolith + realtime sidecar pattern:

- Frontend: static HTML pages per role with vanilla JavaScript.
- Backend API: PHP entry router ('backend/index.php') dispatching to route modules.
- Database: MySQL ('hospital_db') with core schema + runtime ensure/migration helpers.
- Realtime service: Node.js WebSocket server ('realtime/server.js') with HTTP emit bridge.

Data flow:

1. Browser calls REST-like endpoints on 'backend/index.php/<module>/<action>'.
2. Backend authenticates JWT and enforces roles.
3. Backend reads/writes MySQL and logs audit events.
4. Backend emits realtime events to Node sidecar ('/emit').
5. Connected dashboards refresh data through WebSocket + periodic auto-refresh.

`## 3. Technology Stack`

`#### 3.1 Backend`

- Language: PHP (XAMPP environment).
- Dependency manager: Composer.
- Auth library: 'firebase/php-jwt' ^7.0.
- API style: module/action routes through a single front controller.
- Realtime integration: HTTP POST to sidecar ('backend/utils/Realtime.php').

`#### 3.2 Frontend`

- HTML5 + CSS3 + Vanilla JavaScript.
- CSS/JS framework: Bootstrap 5.3.3.
- Icons: Bootstrap Icons 1.11.x.
- Charts: Chart.js 4.4.1 in analytics-heavy dashboards.
- Session storage: 'localStorage' for JWT and user payload.

`#### 3.3 Realtime Service`

- Runtime: Node.js 18+ (project requirement).
- Library: 'ws' ^8.17.0.
- Ports: default HTTP + WebSocket on 8090.
- Interfaces:
 - POST '/emit' for backend event fanout.

3.4 Database

- Engine: MySQL (InnoDB + utf8mb4).
- Bootstrap schema: 'backend/config/database/hospital_db.sql'.
- Runtime schema hardening/extensions: 'backend/utils/DbSchema.php'.

4. Key Functional Modules

- Authentication: login/logout, JWT issuance/validation, session tracking.
- Admin: user management, shifts, settings, logs, monitoring dashboards.
- Reception: patient registration, appointments, queue/check-in, referrals/consents.
- Doctor: waiting list, consultations, prescriptions, tasks/escalations/discharge.
- Nurse: beds, urgent care queues, tasks/escalations/handover, pharmacy request trigger.
- Nurse Aid: triage queue handling, admitted/discharge views, triage analytics.
- Pharmacy: pending prescriptions, dispensing, stock management, advanced pharmacy workflows.
- IT Support: system overview, ticketing, audit search and operational checks.

5. Authentication and Authorization

5.1 Authentication

- Login endpoint: 'auth/login' (POST).
- Token format: JWT (HS256) with configurable issuer/audience/expiration.
- Default token lifetime: 1800 seconds (30 minutes).
- Session model: JWT 'sid' plus database-backed 'user_sessions'/session metadata.

5.2 Authorization

- Route-level role checks through middleware.
- Role normalization handles case/space/hyphen variants.
- Supported roles include:
'admin', 'doctor', 'nurse', 'nurse_aid', 'pharmacist', 'senior_pharmacist', 'receptionist',
'it_support'.

6. API Surface (High-Level)

Routing pattern:

- Base: '/backend/index.php'
- Module dispatch by first URI segment: 'auth', 'admin', 'users', 'reception', 'doctor',
'patients', 'nurse', 'nurse_aid', 'pharmacy', 'inventory', 'reports', 'logs', 'shifts',
'settings', 'it', 'health'.

Representative endpoints (examples):

- Auth: '/auth/login', '/auth/logout'.
- Reception: '/reception/register', '/reception/appointments', '/reception/queue_list'.
- Doctor: '/doctor/waiting_list', '/doctor/complete_refer', '/doctor/analytics'.
- Nurse: '/nurse/beds', '/nurse/assign_bed', '/nurse/pharmacy_request'.
- Nurse Aid: '/nurse_aid/triage_queue', '/nurse_aid/start_triage'.
- Pharmacy: '/pharmacy/pending', '/pharmacy/dispense', '/pharmacy/analytics' and advanced actions.
- Inventory: '/inventory/list', '/inventory/add', '/inventory/update'.
- IT: '/it/overview', '/it/tickets', '/it/ticket_create', '/it/audit'.
- Health: '/health'.

7. Data Model

7.1 Core Tables (bootstrap SQL)

- 'users'
- 'user_sessions'
- 'patients'
- 'medicines'
- 'it_tickets'
- 'visits'
- 'vital_signs'
- 'prescriptions'
- 'billings'
- 'notifications'
- 'contact_inquiries'

7.2 Runtime Extended Tables (DbSchema helpers)

Examples include:

- 'staff_shifts', 'appointments'
- 'reception_handover', 'nurse_handover', 'doctor_handover'
- 'nurse_tasks', 'doctor_tasks', escalation/discharge helper tables
- Pharmacy advanced domain tables: interactions, controlled requests, refills, suppliers, PO/GRN/invoices, quarantine, adjustments, insurance claims
- Audit/activity tables managed by 'ActivityLogger'

8. Realtime and Synchronization

- Frontend connects to 'ws://<hostname>:8090'.
- Backend emits events through HTTP POST to sidecar ('/emit').
- Frontend combines:
 - WebSocket event-triggered refresh.
 - Fixed interval refresh (10s).
 - Cross-tab sync via 'BroadcastChannel' and storage events.

9. Observability and Operations

- Health endpoint returns:
 - API status/version/time.
 - Database connectivity status.
 - Realtime socket reachability status.
- Logging:
 - Activity and audit logs through utility layer.
 - Additional auth/login debug logs under 'backend/logs'.
- Monitoring scripts:
 - 'scripts/monitor_watch.ps1'
 - 'scripts/run_tests.ps1'
 - 'scripts/smoke_test.ps1'

10. Deployment and Environment

Minimum local requirements:

- XAMPP (Apache + PHP + MySQL).
- Node.js 18+ for realtime service.

Baseline local path assumptions:

- 'C:\xampp\htdocs\Hospital_Management_System'

Startup order (local):

1. Start Apache + MySQL (XAMPP).

2. Ensure 'hospital_db' schema imported.
3. Configure 'backend/config/database.php' and 'backend/config/jwt.php'.
4. Start realtime service in 'realtime' ('npm install', 'node server.js').
5. Open 'frontend/pages/auth/login.html'.

11. Security Notes

Current implementation strengths:

- JWT-based auth.
- Role middleware.
- Session invalidation/support logic.

Hardening recommendations:

- Replace default JWT secret immediately.
- Externalize DB credentials and secrets to environment or vault.
- Restrict CORS and backend direct file access in production.
- Remove or protect utility/debug scripts before production release.
- Move to HTTPS-only deployment and secure cookie/session strategy where possible.

12. Known Technical Risks

- Single entry router and dynamic schema expansion may complicate strict change control.
- Some modules rely on runtime table creation; migration discipline is needed for production.
- Realtime sidecar is unauthenticated by default on local endpoint and should be network-restricted.
- Logs may include sensitive operational metadata if not rotated/sanitized.

13. Suggested Next Engineering Steps

1. Introduce versioned database migrations as a mandatory deployment gate.
2. Add OpenAPI/endpoint contract documentation from route modules.
3. Add CI smoke tests for all role dashboards and critical APIs.
4. Add centralized secret management + environment-specific config.
5. Add structured logging sink and retention policy.

14. Source Files Used

- 'README.md'
- 'backend/index.php'
- 'backend/composer.json'
- 'backend/config/database.php'
- 'backend/config/jwt.php'
- 'backend/config/database/hospital_db.sql'
- 'backend/middleware/AuthMiddleware.php'
- 'backend/middleware/RoleMiddleware.php'
- 'backend/routes/auth.php'
- 'backend/routes/health.php'
- 'backend/routes/pharmacy.php'
- 'backend/routes/it.php'
- 'backend/utils/DbSchema.php'
- 'backend/utils/Realtime.php'
- 'backend/services/NotificationService.php'
- 'realtime/package.json'
- 'realtime/server.js'
- 'frontend/assets/js/app.js'
- 'frontend/pages/auth/login.html'

